

From Numbers to Words: How the Model Actually Thinks

SOMA KESARA

PART 2

Our running example is still the same sentence: "**What is my portfolio's exposure to emerging-market equities?**"

Last time, we ended with every word turned into a bundle of numbers carrying **meaning** and **position**. Now we follow those numbers the rest of the way — into the model's "thinking," and back out as the answer that appears on your screen.





Where We Left Off — A One-Line Recap

Each token in our sentence is now a rich list of numbers. The model has *what* each word means and *where* it sits. But it still doesn't know how the words **relate to each other**.

📘 That's the job of the next step — and it's the **single most important idea** in modern AI.

Attention: Letting Every Word Look at Every Other Word

The Problem Attention Solves

Look at our sentence again: *"What is my portfolio's exposure to **emerging-market equities**?"*

The word "**exposure**" is the key word. But "exposure" on its own is vague — exposure to *what*? To sunlight? To risk? To a chemical?

To answer correctly, the model has to figure out that "**exposure**" here is connected to "**emerging-market equities**". Those words, several positions away, give "exposure" its real meaning in this sentence.

📄 This is what **attention** does: it lets every word in the sentence look at every other word and decide which ones matter for understanding it.

The Classroom Analogy

Imagine a classroom of students. Each student is one word in our sentence. The teacher asks every student: *"Look around the room. Who else in here is relevant to you? Pay close attention to them. Ignore the rest."*

"exposure" looks around

Pays a lot of attention to "**portfolio**", "**emerging**", and "**equities**". It barely looks at "is" or "my".

"equities" pays attention

Focuses on "**emerging**" and "**market**" — they describe what kind of equities.

"is" has little to figure out

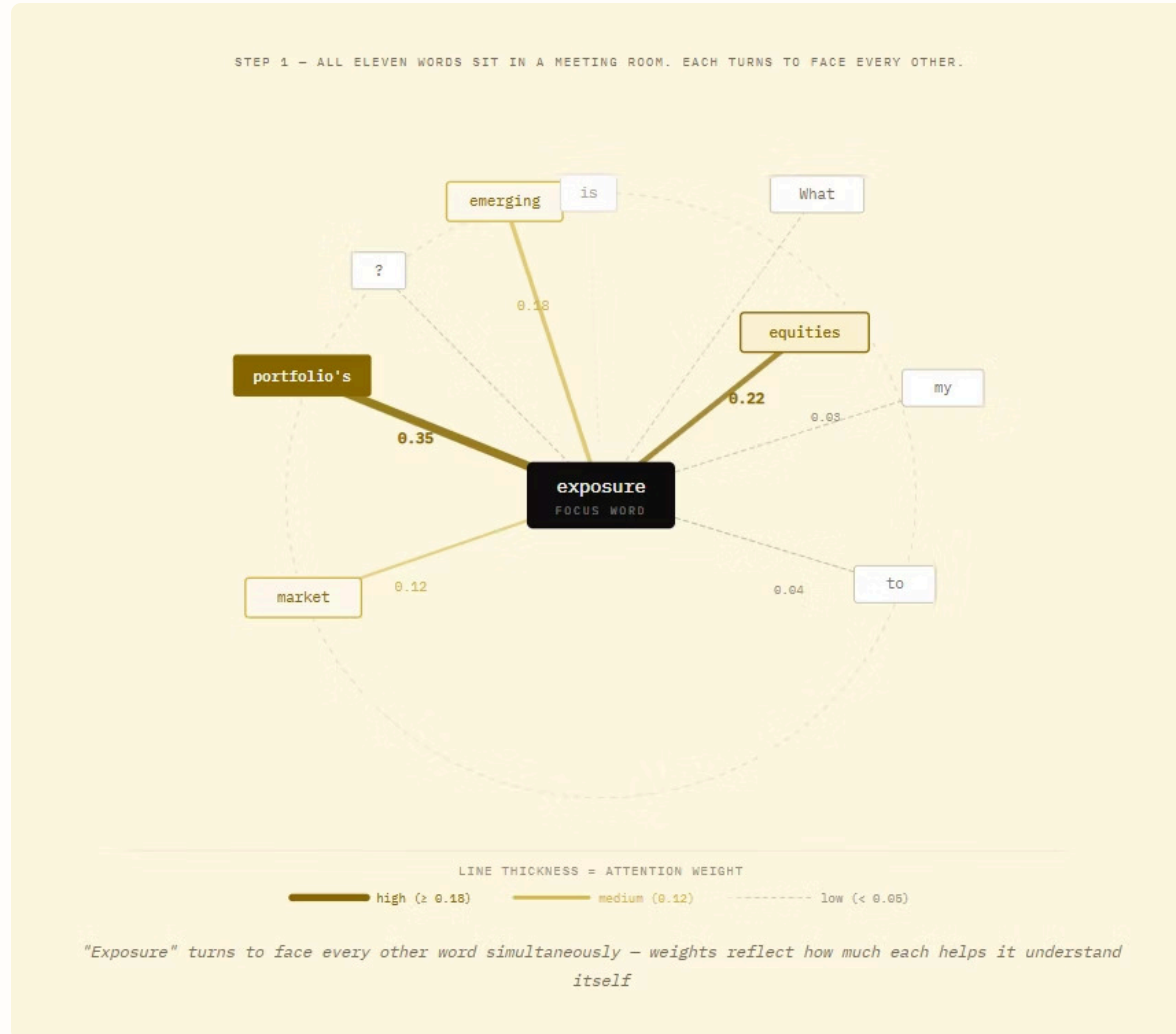
Mostly looks at nothing in particular — it's grammar glue.

Every word does this **at the same time**, in parallel.

That's why GPUs matter — remember the stadium of accountants from Part 1? This is exactly the kind of work they were built for.

Because every word updates its meaning based on context, the model builds a rich, context-aware representation of the whole sentence — not just a bag of isolated word meanings. That is what makes attention far more powerful than earlier approaches like simple word embeddings.

Attention Weights: A Visual



Each word in a sentence attends to every other word with different weights. The lines and connections in the diagram represent those attention scores.

Multi-Head Attention: Many Conversations at Once

WORD	WEIGHT	VISUAL	WHY
"What"	0.02		Question word. Structural, not semantic.
"is"	0.01		Grammar glue. Not informative.
"my"	0.03		Tells me it belongs to someone. Mildly useful.
"portfolio's"	0.35		Highest weight. A portfolio's exposure means financial exposure.
"to"	0.04		Connector. Something I'm exposed to is coming next.
"emerging"	0.18		Relevant. Tells me what kind of thing I'm exposed to.
"market"	0.12		Relevant. Same — qualifies the thing.
"equities"	0.22		Very relevant. The actual thing I'm exposed to.
"?"	0.03		Punctuation. Structural.

Numbers are illustrative — real models learn these from training · weights sum to 1

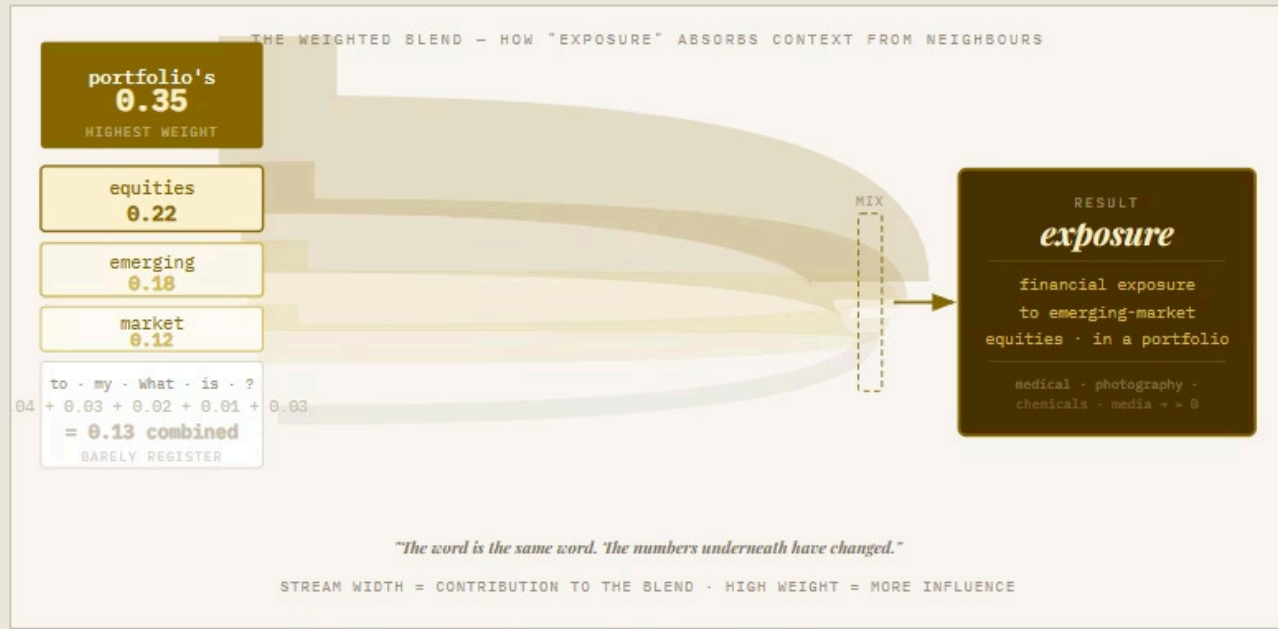
Real transformer models don't run attention just once per layer — they run it in parallel across multiple "heads," each one attending to different kinds of relationships simultaneously.

One head might focus on grammatical structure, another on semantic meaning.

another on long-range dependencies. The outputs of all heads are then combined

Giving the model a richer, multi-dimensional understanding of each word's context.

"Exposure" doesn't just pick the highest-weight word — it takes a **weighted blend** of all their meanings and mixes that into its own vector. High-weight neighbours contribute more; low-weight ones barely register. The result is a new, context-shaped meaning.



WORD BEING PROCESSED

exposure

ITS HIGHEST-WEIGHT NEIGHBOURS

portfolio's	0.35
-------------	------

equities	0.22
----------	------

emerging	0.18
----------	------

market	0.12
--------	------

RESOLVES TO

Financial exposure to emerging-market equities within a portfolio

WORD BEING PROCESSED

equities

ITS HIGHEST-WEIGHT NEIGHBOURS

emerging	0.31
----------	------

market	0.26
--------	------

exposure	0.19
----------	------

portfolio's	0.14
-------------	------

RESOLVES TO

Emerging-market equities, viewed as a category of risk in a portfolio

These two panels ran at the same instant – not sequentially. Every word in the sentence had its own panel.

Weights for "equities" are illustrative; exact values for "exposure" are from the article.

EVERY WORD ATTENDS TO EVERY OTHER WORD - SIMULTANEOUSLY



ATTENTION LAYER

Every word simultaneously reads every other word and updates its meaning

NO SEQUENCE - NO WAITING - FULLY PARALLEL

OUTPUT - ENRICHED



"Financial exposure,
specifically to emerging-market equities
within the context of a portfolio."

LAYERS

Layer N

Layer N-1

Layer N-2

Layer N-3

Layer 1

repeats >N

ARCS ABOVE INPUT ROW SHOW ATTENTION FROM "EXPOSURE" ONLY - ALL OTHER WORDS DO THE SAME SIMULTANEOUSLY

Why It's Called "Attention"

Because each word literally **pays more attention** to some words and less to others. The model assigns a weight — a number between 0 and 1 — to every pair of words.

A simplified picture of what **"exposure"** pays attention to in our sentence:

Word it's looking at	Attention Weight
"portfolio"	0.35
"equities"	0.25
"emerging"	0.20
"market"	0.15
"is", "my", "to", "?" (combined)	0.05

 These weights tell the model: *"When you're updating what 'exposure' means in this sentence, mostly use information from 'portfolio' and 'equities'."*



Every Word, Every Other Word — All at Once

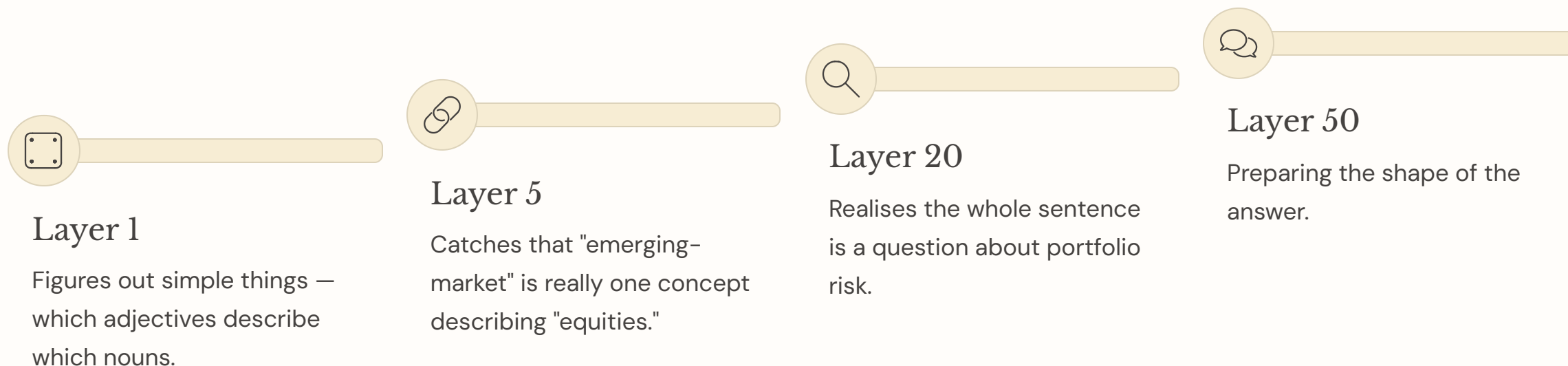
Here's the part that surprised researchers when this idea was introduced in 2017 (the famous paper was called "**Attention Is All You Need**"):

You don't need anything fancy. You don't need the model to read the sentence left-to-right like we do. You just let **every word look at every other word, all at once**, and figure out what to focus on.

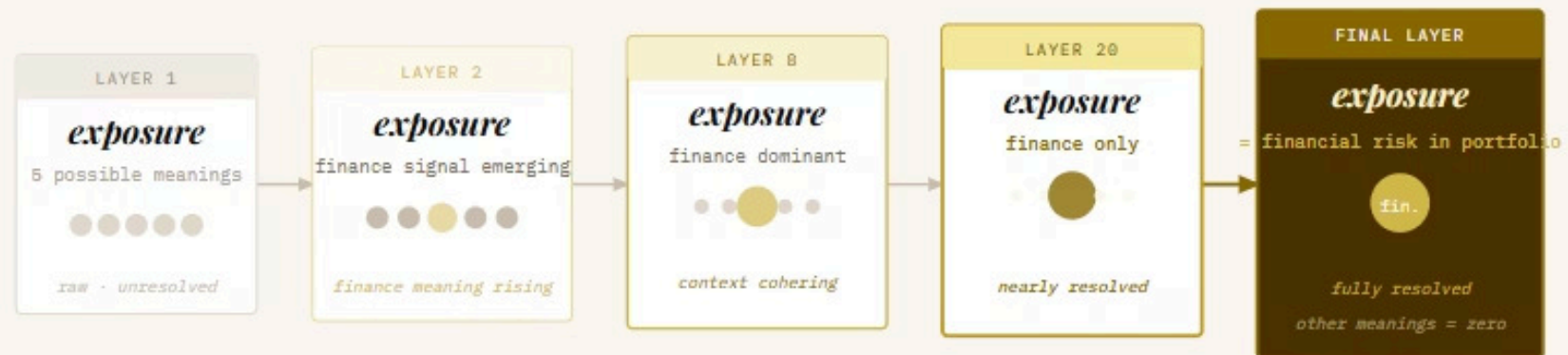
- ✅ That single idea is the foundation of every modern LLM — ChatGPT, Claude, Gemini, all of them.

Many Layers of Attention, Not Just One

One round of attention isn't enough. Real models do this **many times in a row** — typically 30 to 100 layers stacked on top of each other.



📌 **The big idea:** After all these layers, the numbers representing "exposure" aren't just the generic meaning of "exposure" anymore. They've been reshaped to mean *"exposure, in the specific context of a portfolio question about emerging-market equities."*



SAME WORD "EXPOSURE" - EACH COLUMN = ONE ATTENTION LAYER - MEANING SHARPENS WITH EACH PASS

Dots represent the 5 possible meanings - Size = current probability weight

THE SENTENCE - "SHE" MUST RESOLVE TO ONE OF THESE PEOPLE

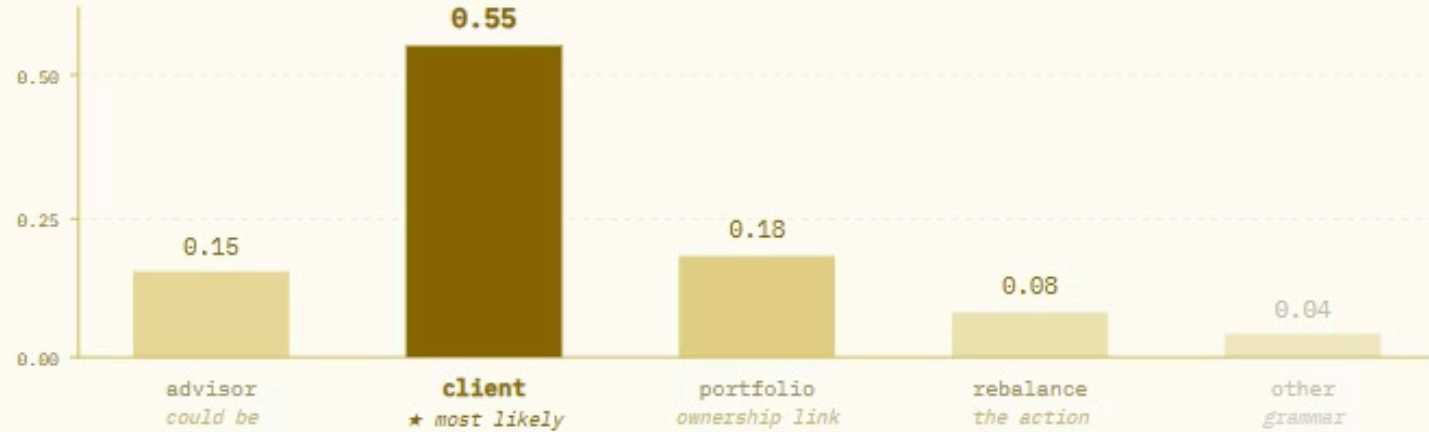
*"The **advisor** told the **client** that she needed to rebalance her **portfolio**."*

candidate A

candidate B ★

↑ ambiguous

ATTENTION WEIGHTS - "SHE" LOOKING AT EACH WORD

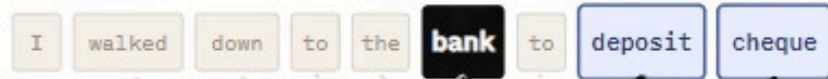


"She" resolves to "the client" — not by rule, but by the weight of context.

AMBIGUITY RESOLVED - NO GRAMMAR RULE REQUIRED

SENTENCE A

"I walked down to the bank to deposit the cheque."



→ deposit
→ cheque

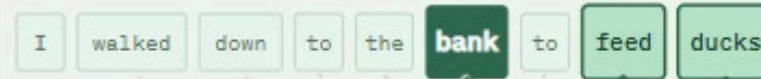
bank = financial institution

FINANCE CONTEXT DOMINATED

"deposit" and "cheque" are strongly financial.
Nothing in the sentence points to water.

SENTENCE B

"I walked down to the bank to feed the ducks."



→ feed
→ ducks

bank = riverside

NATURE CONTEXT DOMINATED

"feed" and "ducks" point outdoors.
Nothing in the sentence points to finance.

The word "bank" is identical in both sentences.

Its neighbours determine its meaning.

NO RULE WAS EVER WRITTEN · LEARNED FROM CONTEXT

STEP 9 — Predicting the Next Word

What the Model Is Actually Doing

Here is the part most people find genuinely surprising:

An LLM is not answering your question. It's predicting the next word.

That's it. That's the whole game.

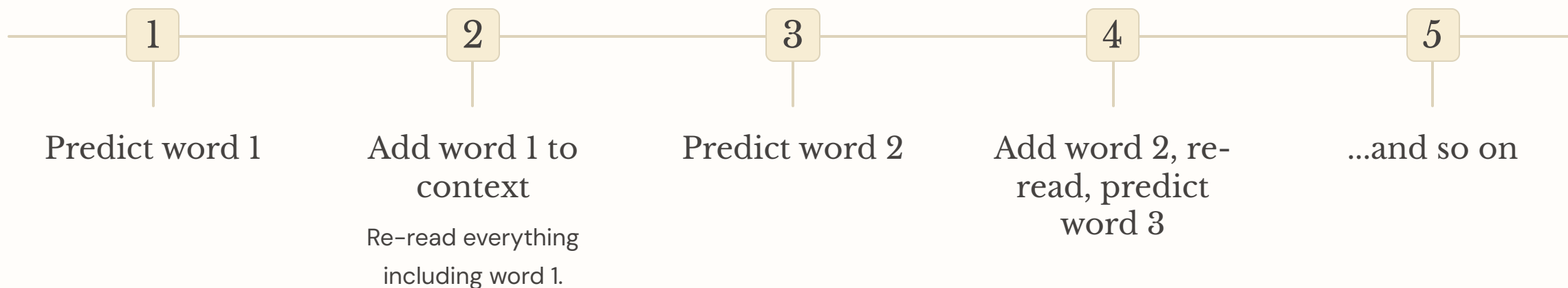
After all those layers of attention, the model arrives at one job: **"Given everything I've just processed, what is the most likely next word?"**

Then it does that again. And again. And again. One word at a time, until the answer is complete.

This Is Why Answers Stream in Word by Word

Remember the question from Part 1 — *why does it reply one word at a time instead of returning the whole answer instantly?*

This is the answer. The model **literally cannot** produce the whole reply at once, because each new word depends on all the words it has already written.



It's like a writer who can only see what they've written so far and has to decide the next word based on everything before it.

Probability Distributions: Not One Answer, But a Ranked List

The Model Doesn't Pick One Word — It Scores All of Them

Here's the next surprise. When the model predicts the next word, it doesn't say *"the answer is 'your'."* It produces a **score for every single word in its vocabulary** — all 100,000 or so of them.

What a probability distribution looks like

Possible next word	Probability
"Your"	42%
"Based"	18%
"Looking"	12%
"I"	8%
"Currently"	5%
"The"	4%
...	...
"banana"	0.0001%
"xylophone"	0.00001%

What this means

This ranked list is called a **probability distribution**. It says: *"Here is how likely I think each possible next word is."*

The model scores all 100,000+ words in its vocabulary every single time it needs to produce one token of output.

 The word it eventually picks is one choice from this big ranked list — never a certainty.



Why This Matters

The model is **never** certain. Even when it sounds confident, underneath it's always working with probabilities. The word it eventually picks is one choice from this big ranked list.

This is the root of one of the questions from Part 1:

"Why does it sometimes give a wrong answer with complete confidence?"



Confidence in the words $\neq$ Confidence in the facts. The model picked the next word that *sounded right* based on patterns it learned. It has no built-in fact-checker telling it whether the resulting sentence is true. If millions of training documents used confident-sounding language for a particular topic, the model learned to sound confident — even when it's wrong.

STEP 11 — How the Model Picks One Word: Sampling

So we have a ranked list of 100,000 possible next words. Which one does the model actually use? This is called **sampling**, and it's where things get interesting — because there are several different strategies, and they each change the personality of the answer.

Strategy 1: Greedy — Always Pick the Top One

The simplest strategy: just pick the highest-probability word every time.

Pros

- Predictable and consistent
- Same question → same answer

Cons

- Boring and repetitive
- If you ask the same question twice, you get the exact same reply word-for-word
- The "safest" word isn't always the best one — sometimes the 2nd or 3rd choice would have made a much better sentence

Strategy 2: Temperature — Adjusting How Adventurous the Model Is

Temperature is a dial that controls how willing the model is to pick lower-ranked words. Think of it as a creativity setting.

Temperature	What it does	Feels like
0.0	Always pick the top word (greedy)	Robotic, repetitive
0.3	Slight randomness, mostly safe choices	Careful, factual
0.7	Balanced — the everyday default	Natural, varied
1.0	More adventurous, considers more options	Creative, sometimes off-base
1.5+	Wild — picks unusual words often	Weird, unpredictable

Low Temperature

Like a cautious analyst who only says what they're sure of. Ideal for a **wealth-management assistant** — use 0.2–0.4 for consistency and accuracy.

High Temperature

Like a brainstorming partner who throws out unusual ideas — sometimes brilliant, sometimes nonsense. Use 0.9+ for **writing poems** or **brainstorming product names**.



Strategy 3: Top-K — Only Consider the Top K Options

Even with temperature, the model could occasionally pick an absurd word from far down the list. **Top-K** prevents that.

 **Top-K = 5** means: *"Only look at the top 5 most likely words. Ignore everything else. Pick from those 5."*

So instead of choosing from 100,000 candidates, the model chooses from a tiny shortlist. The exact word picked depends on temperature, but it's guaranteed to come from a sensible top group.

Common values: Top-K of 40 or 50 is typical.

Strategy 4: Top-P (Nucleus Sampling) — Adaptive Shortlist

Top-K has one weakness: sometimes only **2 words** are sensible, and sometimes **50 words** are all reasonable. A fixed K doesn't adapt.

Top-P fixes this. Instead of "pick from the top 5," it says: *"Add up the top words until their combined probability reaches P. Pick from that group."*

Top-P = 0.9

Take the smallest set of top words whose probabilities sum to 90%.

High confidence scenario

If one word has 92% probability, the shortlist is just that one word.

Spread-out scenario

If probabilities are spread out, the shortlist might include 30 words.

✓ This is the **most popular strategy in modern LLMs** because it adapts to how confident the model is.

How They Combine in Practice

Real systems usually combine these strategies together:

```
temperature = 0.7  
top_p = 0.9  
top_k = 40
```

Read it as: *"From the top 40 words whose combined probability covers 90% of the model's confidence, pick one — with a moderate level of randomness."*

📌 **The trade-off in one line:** lower temperature and tighter top-K/top-P = more accurate but boring; higher = more creative but riskier.

Detokenisation: Turning Numbers Back into Text

Remember Part 1, Step 4 — the moment text became numbers and the model never saw text again? Now we have to reverse that.

The Model Outputs a Token ID, Not a Word

When the model picks its next word, it doesn't output the word "Your." It outputs a **token ID** — a number, like 7927. That number has to be looked up in the same fixed vocabulary dictionary from Part 1, and converted back to the text fragment it represents.

Token ID	Text piece
7927	"Your"
19212	" portfolio"
706	" has"
220	" "
868	"12"
13	"."
20	"4"
4	"%"

Why This Looks Weird Sometimes

Recall: tokens aren't always whole words. So when the model is producing an answer, it might output tokens like:

```
"Your" · " portfolio" · " has" · " 12" · "." · "4" · "%" · " exposure" · " to" · " emerging" · "-" · "market" · " equities" · "."
```

Strung together, that becomes:

"Your portfolio has 12.4% exposure to emerging-market equities."

Notice the percentage 12.4% came out as **four separate tokens** (" 12", ".", "4", "%"). The model assembled the number piece by piece — it doesn't "know" numbers as numbers, it just predicts each piece in sequence.

⚠️ **This is also why LLMs sometimes make arithmetic mistakes.** They're not calculating — they're predicting the next digit-token based on patterns. Ask one to multiply two large numbers and it might confidently produce a wrong answer because it's pattern-matching, not computing.

Streaming vs. All-at-Once

Two Ways to Send the Answer Back to You

Once the model starts producing tokens, the system has a choice about how to deliver them to your screen.

Streaming (the default in ChatGPT, Claude, Gemini)

Each token is sent to you **the moment it's produced**. You see the answer appear word-by-word, almost like watching someone type in real time.



Start reading immediately

No waiting — you begin reading as soon as the first word appears.



Feels alive and responsive

The experience feels dynamic and engaging, like a real conversation.



Stop early if needed

If the answer is going wrong, you can stop it early and save time.

Under the hood: Each token is sent over the internet as soon as the GPU produces it. The connection stays open the whole time, like a phone call rather than a letter.



All-at-Once (Batch Mode)

The system waits until the **entire answer is finished**, then sends it as one big chunk.

When it's used:

- API calls where the receiving programme just wants the final text
- Automated workflows that don't have a human waiting
- When you need to do post-processing on the full answer before showing anything



Trade-off: You wait longer for the first thing to appear, but it can be slightly simpler to handle on the receiving end.



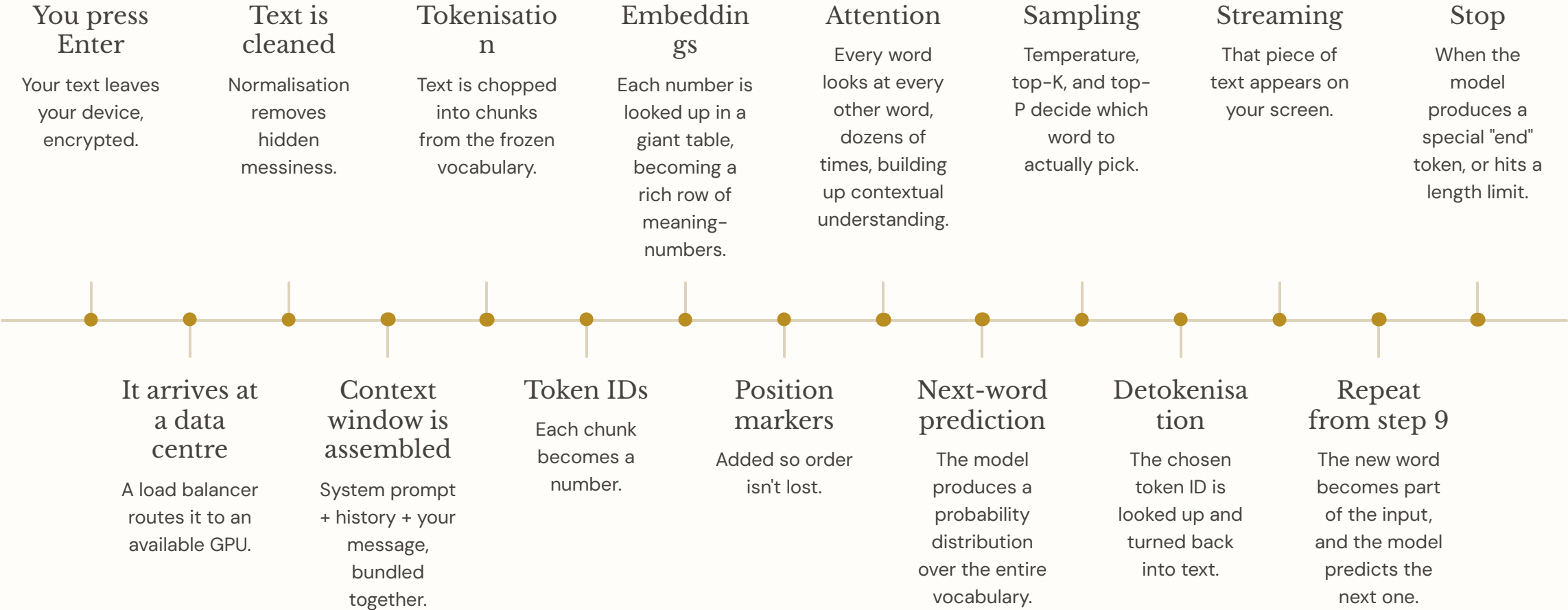
Why Streaming Doesn't Make It Faster

A common misconception: streaming feels faster, but the **total time is roughly the same**. The model is producing tokens at the same speed either way. Streaming just shows you each token as it's made, instead of holding them all back until the end.

It's the difference between a chef bringing each dish to your table as it's cooked, versus waiting until the whole meal is ready and bringing it all out together. Same total cooking time — very different experience.

Bringing It All Together — The Full Journey of Our Request

Here's the complete pipeline from Part 1 + Part 2:



✔ All of this happens in a few seconds. For each word.

Callbacks — The Questions from Part 1, Answered

We promised to come back to the open questions. Here's where we stand now:

"Why does it reply one word at a time?"

Because each word is predicted from everything that came before it. The model literally cannot produce the whole answer at once — it has to feed its own output back in, word by word.

"Why does it sometimes give wrong answers with complete confidence?"

Because it's predicting words that *sound* right based on patterns, not facts. Confidence in the language doesn't mean confidence in the content. There's no fact-checker built in.

"Why does running these models need so much computing power?"

Because of attention. Every word looking at every other word, across dozens of layers, with thousands of meaning-numbers each — that's billions of multiplications for every single token of output.

"Does it really know English, Spanish, and Hindi?"

It doesn't "know" any language the way we do. It learned which tokens tend to appear near which other tokens. The patterns of Hindi look different from English in the numbers — and the model captured both sets of patterns.

What's Still Open

Two questions remain — and they're the deepest ones:

Does it truly understand you?

Or does it just process numbers? The distinction between genuine understanding and sophisticated pattern-matching remains unresolved.

How is it like — or unlike — how we think?

If it's predicting the next word, what does that share with human cognition?
What is fundamentally different?

❓ These don't have settled answers, even among researchers. They're worth sitting with as you use these tools every day.



How Attention Actually Works

The Mechanism Behind Every Modern AI System

We've referenced **attention** throughout the series. It's time to slow down and look at it properly — because once you see how it works, you understand most of what's actually happening inside an LLM.

 No history. No maths. Two worked examples, in plain language.



The One-Sentence Version

Attention is every word in a sentence looking at every other word, deciding which ones matter for understanding itself, and updating its meaning accordingly — all at the same time.

That's it. The rest of this document is just unpacking that sentence carefully, with examples.

The Setup

Recall from Part 1: by the time the model is ready to "think," every word in our sentence has been turned into a long row of numbers. That row captures the word's general meaning — the dictionary version of it.

But the dictionary version isn't enough. Take the word "**exposure**." On its own, it could mean:

Medical

Exposure to sunlight

Safety

Exposure to a chemical

Photography

Exposure in a photograph

Finance

Exposure to risk

Media

A journalist's exposé

📌 The model needs to figure out *which kind* of exposure this is, based on the rest of the sentence. That's the job attention does. It takes each word's generic meaning and **sharpens it using context**.

Example 1: Our Running Sentence

Let's walk through it carefully, on our familiar example:

"What is my portfolio's exposure to emerging-market equities?"

Eleven words. The model is going to do attention on all of them in parallel, but to keep things readable, let's follow just one of them: **"exposure."**





Step 1: "Exposure" Looks Around the Room

Imagine all eleven words sitting in a meeting room. The model gives every word the same instruction:

"Turn to every other word in the room. Decide how relevant each one is to understanding yourself."

"Exposure" turns to each of the other ten words and asks itself: *how much does this word matter for understanding me, right here, in this sentence?*

Step 2: "Exposure" Assigns a Weight to Each Word

It scores each one — a number between 0 and 1 — reflecting how much attention to pay:

Word "exposure" is looking at	How relevant?	Why
"What"	0.02	Question word. Tells me this is a question, but doesn't tell me what kind of exposure.
"is"	0.01	Grammar glue. Not informative.
"my"	0.03	Tells me it belongs to someone. Mildly useful.
"portfolio's"	0.35	Highly relevant. This is what I belong to. A portfolio's exposure means financial exposure.
"to"	0.04	Connector. Tells me there's something I'm exposed to coming next.
"emerging"	0.18	Relevant. Tells me what kind of thing I'm exposed to.
"market"	0.12	Relevant. Same — qualifies the thing.
"equities"	0.22	Very relevant. This is the actual thing I'm exposed to.
"?"	0.03	Punctuation. Structural.

(Numbers are illustrative — a real model produces these values from training, and they sum to 1.)

Step 3: "Exposure" Updates Its Meaning

Now "exposure" takes a **weighted blend** of all those words' meanings, using these weights, and mixes it into its own.

The result: "exposure" is no longer the generic dictionary word with five possible meanings. It has been **coloured** by sitting near "portfolio's," "equities," "emerging," and "market." Its meaning shifts toward:

"Financial exposure, specifically to a category of equity investments, within the context of a portfolio."

The other four meanings — sunlight, chemicals, photography, journalism — fade away. They got almost no attention weight, because nothing in the sentence supports them.

✓ The word is the same word. The numbers underneath have changed.

What Happened in Parallel

While "exposure" was doing this, **every other word in the sentence was doing exactly the same thing, at the same instant.**

"Equities" looked around

Paid most attention to "emerging" and "market" (they describe what kind of equities) and to "exposure" (because it's the thing being asked about). It came out meaning *"emerging-market equities, viewed as a category of risk in a portfolio."*

"My" looked around

Paid most attention to "portfolio's" — because "my" refers to the speaker's portfolio. It came out coloured as a possessive linked to that specific portfolio.

"Emerging" and "market" looked around

Both locked onto "equities" — they're modifiers describing it. They came out tightly bound to "equities" as a single concept.

"What" looked around

Found very little to attend to strongly — it's a question word, mostly contributing structure rather than meaning.



One Layer Done — Dozens More to Go

Every word's meaning has now been **enriched by every other word's meaning**, weighted by relevance. The sentence isn't just a sequence of words anymore. It's a network of meanings that have all influenced each other.

That's one round — one **layer** — of attention.

- ❗ In a real model, this happens dozens of times in a row. Each layer takes the enriched words from the layer below and enriches them further. By the top of the stack, every word's meaning has absorbed context from the entire sentence, dozens of times over.

Example 2: Why Attention Is So Powerful

Let's try a trickier sentence:

"The advisor told the client that she needed to rebalance her portfolio."

There's a question hiding in this sentence: **who does "she" refer to?** Is it the advisor, or the client? In English, this is genuinely ambiguous from grammar alone.

Watch what attention does when it gets to the word **"she."** Its job: figure out which person in this sentence it refers to.

"She" looking at	Weight	Reasoning
"advisor"	0.15	Could be — also a person, also female-possible.
"client"	0.55	Much more likely — the client owns the portfolio in this sentence.
"portfolio"	0.18	Relevant — connects "she" to whose portfolio it is.
"rebalance"	0.08	The action being recommended.
Other words	0.04	Mostly grammar.

✔ "She" comes out the other side meaning specifically *"the client, who is being told to rebalance her portfolio."* The ambiguity has been resolved — not by a hard rule, but by the model weighing the context.

Example 3: The Same Word, Two Different Sentences

Here's a final example that shows attention at its most powerful. Consider the word "**bank**" in these two sentences:

Sentence A

"I walked down to the bank to deposit the cheque."

"Bank" looks around. High-attention neighbours: "**deposit**" and "**cheque**." These strongly suggest the financial meaning.

Result: **bank-the-institution**.

Sentence B

"I walked down to the bank to feed the ducks."

"Bank" looks around. High-attention neighbours: "**ducks**" and "**feed**." These suggest something outdoorsy and natural.

Result: **bank-the-riverside**.

📄 The model never had to be told the rule *"if bank is near deposit, it means financial; if bank is near ducks, it means riverside."* It learned, from billions of examples, what each word tends to attend to in what contexts. Attention plus training takes care of the rest.

This is why modern AI systems handle ambiguity so much better than earlier ones. Every word can re-shape itself based on its company.

Why This Matters for What You Do with These Tools

If you've followed the three examples, you now have a working intuition for the most important idea in modern AI. A few practical implications worth carrying with you:

Context is not optional — it's the entire mechanism

When you give an LLM a vague prompt, you're not just being unclear. You're starving the attention mechanism of the neighbours it needs to sharpen the meaning of each word. *"Write me something about exposure"* gives the model almost nothing to work with. *"Write a one-paragraph note for a client about their portfolio's exposure to emerging-market equities, in our firm's professional tone"* gives every important word a roomful of neighbours to attend to.

More relevant context is usually better than more context

Attention can pay attention to anything in the room. But noise in the room competes for that attention. Putting irrelevant documents into a RAG prompt doesn't just waste tokens — it dilutes the attention the model can give to the relevant material.

More Practical Implications

 Position matters less than it used to, but still matters

Attention treats all positions equally in principle. But in practice, models pay slightly more attention to the start and end of long contexts than the middle. Important instructions at the start of a prompt; important questions at the end. Don't bury the key request in the middle of a wall of text.

 Long context is genuinely powerful, not just a vanity number

When a vendor says their model handles a million tokens of context, they're saying every word in that million-token document can look at every other word and pull in relevant meaning. That's the same mechanism that resolves "she" to "client" in a one-sentence example — just scaled up enormously.

The One-Paragraph Version

Every word in a sentence enters a kind of meeting room. Each word looks at every other word in the room and assigns a weight reflecting how relevant that word is to understanding itself. Then it updates its own meaning by mixing in a weighted blend of the others. All words do this simultaneously, in parallel — no waiting, no sequence. The result is that every word's meaning has been sharpened by its context. "Exposure" sitting next to "portfolio" and "equities" comes out meaning financial exposure. "She" sitting next to "client" comes out referring to the client. "Bank" sitting next to "ducks" comes out meaning a riverside. This happens dozens of times in stacked layers, so every word ends up absorbing context from across the entire input. That's attention. That's the engine.

- ✔ Once you see it, you can't unsee it. Every coherent answer a modern AI gives, every connection it draws across a long document, every time it picks the right interpretation of an ambiguous word — that's attention doing its work, quietly, billions of times a second.